

b) Investigación

- HTTP

¿Qué es HTTP y como funciona? ¿Cuáles son los métodos de petición y que realiza cada uno? ¿Qué diferencia existen entre una Petición GET y POST? ¿Cómo es el envío de parámetros, qué significa “=” y que significa “&” cuando se envían parámetros por POST y GET? ¿Cuáles son los códigos de estado en HTTP?

HTTP (HyperText Transfer Protocol) es el protocolo que usan los navegadores y los servidores web para intercambiar información. Sigue un modelo cliente-servidor de petición-respuesta:

1. El cliente (navegador, app, curl) abre una conexión con el servidor.
2. Envía una petición (request) que contiene: una línea inicial con el método, la ruta y la versión (GET /index.html HTTP/1.1), unas cabeceras (como por ejemplo Content-Type) y opcionalmente un cuerpo.
3. El servidor procesa la petición y devuelve una respuesta (response) con: una línea de estado (HTTP/1.1 200 OK), cabeceras y un cuerpo (HTML, JSON, imagen, etc.).

HTTP es sin estado (stateless): cada petición es independiente y el servidor no recuerda las anteriores. Para mantener sesiones se usan mecanismos adicionales como las cookies o los tokens.

Peticiones:

- GET: Solicita un recurso. Solo lee datos, no debería modificar nada.
- POST: Envía datos al servidor para crear un recurso o procesar algo (formularios, registros).
- PUT: Reemplaza un recurso existente (o lo crea si no existe).
- PATCH: Modifica un recurso.
- DELETE: Elimina un recurso.
- HEAD: Igual que GET pero devuelve solo las cabeceras, sin cuerpo.
- OPTIONS: Consulta qué métodos permite el servidor para un recurso.
- TRACE: Devuelve la petición tal como el servidor la recibió (diagnóstico).
- CONNECT: Establece un túnel, típicamente para proxies y HTTPS.

GET se utiliza para obtener datos, POST para enviarlos o crearlos. En una petición GET, los parámetros viajan en la URL y son visibles en ella, mientras que en POST viajan en el cuerpo y no son visibles. Además GET es idempotente, que significa que repetir la petición no afecta en nada, pero repetir una petición POST puede crear duplicados.

Los parámetros se envían en formato clave=valor, = separa el nombre del parámetro de su valor, & separa un parámetro del siguiente.

Los códigos de estado HTTP son números de tres cifras que el servidor incluye en la respuesta para indicar el resultado. Se agrupan por su primer dígito:

- Los que empiezan con 1 son informativos, por ejemplo 100 Continue (el cliente puede seguir enviando el cuerpo).
- Los que empiezan con 2 son de éxito (200 OK: petición correcta, 201 Created: recurso creado, 202 Accepted: aceptada, pero aún se procesa, 204 No Content: éxito, sin cuerpo en la respuesta)
- Los que empiezan con 3 son de redirecciones (301 Moved Permanently: el recurso cambió de URL para siempre, 302 Found: redirección temporal, 304 Not Modified: el recurso no cambió, usa tu copia en caché, 307 / 308: redirecciones temporal / permanente que conservan el método)
- Los que empiezan con 4 son de errores del cliente (400 Bad Request: petición mal formada, 401 Unauthorized: falta autenticación, 403 Forbidden: autenticado, pero sin permisos, 404 Not Found: el recurso no existe.)
- Los que empiezan con 5 son de errores del servidor (500 Internal Server Error: fallo genérico del servidor, 501 Not Implemented: funcionalidad no implementada, 502 Bad Gateway: respuesta inválida de un servidor intermedio).

- REST-API

¿Qué es REST y para qué sirve? ¿Qué es una API, para que sirve y cual es el objetivo de combinarlo con REST? ¿Qué es un Endpoint?. Realizar un ejemplo Gráfico del Uso de API REST.

Una **API** (Application Programming Interface) es un conjunto de definiciones, reglas y protocolos que permite que dos o más sistemas de software se comuniquen e intercambien datos entre sí de manera estandarizada, sin necesidad de conocer los detalles de implementación interna del otro. Sirve para desacoplar componentes, reutilizar lógica de negocio y exponer funcionalidades de forma controlada y segura.

REST (Representational State Transfer) es un estilo arquitectónico para sistemas distribuidos basado en la web. Sirve para diseñar servicios de red predecibles, ligeros y escalables apoyándose en los principios y protocolos nativos de la web (principalmente HTTP). Se rige por seis restricciones formales:

- **Cliente-Servidor:** Separación estricta de responsabilidades.

- **Stateless (Sin estado):** Cada petición del cliente al servidor debe contener toda la información requerida; el servidor no guarda contexto de sesión del cliente entre requests.
- **Cacheable:** Las respuestas deben definirse explícitamente como almacenables o no en caché para optimizar el rendimiento de red.
- **Interfaz uniforme:** Identificación de recursos vía URIs, manipulación a través de representaciones (JSON/XML), mensajes autodescriptivos e hipermedios como motor del estado de la aplicación.
- **Sistema en capas:** El cliente no necesita saber si está conectado directamente al servidor final o a intermediarios.
- **Código bajo demanda:** El servidor puede transferir temporalmente lógica ejecutable al cliente.

Una API por sí misma no impone ningún estándar de comunicación. Al combinar una API con los principios REST (**API REST**), se obtiene una interfaz web estándar donde:

1. Las entidades del negocio se modelan como **recursos** (sustantivos).
2. Las operaciones sobre esos recursos se ejecutan usando los **métodos HTTP nativos**:
 - GET: Obtener un recurso (seguro e idempotente).
 - POST: Crear un nuevo recurso.
 - PUT: Reemplazar completamente un recurso (idempotente).
 - PATCH: Modificar parcialmente un recurso.
 - DELETE: Eliminar un recurso (idempotente).
3. Se utilizan **códigos de estado HTTP** universales (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error).

Un **Endpoint** es el punto final de contacto expuesto por el servidor al cual el cliente envía sus peticiones HTTP para acceder o manipular un recurso específico. Técnicamente se compone de una **URL** combinada con un **método HTTP**.

Ejemplo:

GET https://api.tienda.com/v1/productos → Endpoint para listar el catálogo.

POST https://api.tienda.com/v1/productos → Endpoint para crear un producto.

Mencione cuales son los patrones de diseño que se aplican para API. Realizar una explicación de cada uno de ellos.

1. API Gateway

Utilizamos un componente intermedio como punto único de entrada para todas las peticiones externas. Se encarga del enrutamiento hacia los microservicios internos,

terminación SSL, autenticación/autorización, balanceo de carga, monitoreo y limitación de tasa.

2. BFF (Backend For Frontend)

Se implementa un backend intermedio específico por cada tipo de cliente (un BFF para iOS/Android y un BFF para Web). Cada BFF consume las APIs internas del dominio y entrega al cliente final únicamente los datos y formatos adaptados a su pantalla y ancho de banda.

3. Facade (Fachada)

Se expone un endpoint REST simplificado y coherente que oculta toda la complejidad interna. El cliente solo interactúa con la fachada, la cual orquesta las llamadas subyacentes.

4. Circuit Breaker (Disyuntor)

Si las llamadas al servicio destino fallan recurrentemente por encima de un umbral, el circuit breaker se "abre" inmediatamente, rechazando peticiones entrantes de forma inmediata o devolviendo una respuesta por defecto sin esperar al timeout, permitiendo que el servicio downstream se recupere.

5. Data Transfer Object (DTO)

Se definen objetos planos sin lógica de negocio exclusivamente dedicados al transporte de datos entre el cliente y el controlador de la API. Las entidades internas se transforman a DTOs de salida, y las entradas del cliente se validan mediante DTOs de petición.

6. CQRS (Command Query Responsibility Segregation)

Se separan las rutas y modelos de la API en dos canales:

- Commands (Escritura): Endpoints que alteran el estado (POST, PUT, DELETE), enfocados en validación estricta y lógica de negocio.
- Queries (Lectura): Endpoints que solo recuperan datos (GET), optimizados para velocidad, vistas denormalizadas y almacenamiento en caché.

7. Outbox Pattern

La API guarda la entidad de negocio y el evento a emitir en una misma transacción relacional dentro de una tabla outbox. Un proceso desacoplado lee los eventos no procesados de la tabla y los publica al broker garantizando entrega de al menos una vez (at-least-once delivery).

-JSON:

JSON (*JavaScript Object Notation*) es un formato ligero de intercambio de datos. Su propósito principal es permitir la transmisión estructurada de información entre un servidor y una aplicación web o servir como archivo de configuración en diversos sistemas informáticos.

El formato se compone principalmente de dos estructuras de datos:

- **Objeto:** Una colección desordenada de pares clave-valor, delimitada por llaves. Las claves son siempre cadenas de texto entre comillas dobles.
- **Array (Colección):** Una lista ordenada de valores, delimitada por corchetes.

Objeto en Java:

```
Persona p = new Persona("Ana", 28, true);
```

Equivalente en JSON:

```
{  
  "nombre": "Ana",  
  "edad": 28,  
  "activo": true  
}
```

Colección de objetos en Java:

```
List<Persona> equipo = Arrays.asList(new Persona("Ana", 28, true), new  
Persona("Carlos", 32, false));
```

Equivalente en JSON:

```
[  
  {  
    "nombre": "Ana",  
    "edad": 28,  
    "activo": true  
  },  
  {  
    "nombre": "Carlos",  
    "edad": 32,  
    "activo": false  
  }  
]
```

```
{  
  "nombre": "Carlos",  
  "edad": 32,  
  "activo": false  
}  
]
```

Alternativas a JSON

- **XML:** Un formato más antiguo y estructurado mediante etiquetas jerárquicas; ideal para integraciones empresariales complejas, aunque más pesado de procesar.
- **YAML:** Un formato sumamente legible y limpio enfocado en la indentación, ampliamente utilizado en archivos de configuración (Docker, Kubernetes).
- **Protocol Buffers (Protobuf):** Un formato binario compacto y de alto rendimiento desarrollado por Google, perfecto para la comunicación interna entre microservicios.
- **CSV:** Un formato de valores separados por comas, excelente para almacenar datos tabulares sencillos o realizar migraciones hacia bases de datos.

-POSTMAN:

Postman es una plataforma de colaboración y desarrollo de APIs ampliamente utilizada por desarrolladores y evaluadores de software. Permite diseñar, construir, probar, documentar y simular APIs de forma visual sin necesidad de escribir código adicional para hacer peticiones HTTP.

Sitio oficial: www.postman.com

1. GET

Se utiliza para solicitar información al servidor.

- **En Postman:**
 1. Selecciona el método **GET** en el menú desplegable junto a la barra de URL.
 2. Ingresa la URL:
https://api.ejemplo.com/v1/usuarios.
 3. Hace clic en **Send**.

2. POST

Se utiliza para enviar nuevos datos al servidor.

- **En Postman:**

1. Selecciona el método **POST**.
2. Ingresa la URL:
https://api.ejemplo.com/v1/usuarios.
3. Ve a la pestaña **Body**, marca la opción **raw** y selecciona el formato **JSON**.
4. Agrega los datos del nuevo usuario:

```
{  
  
  "nombre": "Lucía",  
  
  "edad": 25  
}
```

5. Hace clic en **SEND**

3. PUT

Se utiliza para modificar un recurso existente o crearlo si no existe, reemplazándolo en su totalidad.

- **En Postman:**

1. Selecciona el método **PUT**.
2. Ingresa la URL apuntando a un registro específico:
https://api.ejemplo.com/v1/usuarios/1.
3. En la pestaña **Body** -> **raw** (JSON), ingresa los nuevos datos completos:
4. Hace clic en **SEND**

4. DELETE

Se utiliza para borrar un recurso específico del servidor.

- **En Postman:**

1. Selecciona el método **DELETE**.
2. Ingresa la URL del recurso a eliminar:
https://api.ejemplo.com/v1/usuarios/2.
3. Haz clic en **Send**.